

Structural Vibration Analyzer

Full FFT Analyzer Documentation

Contents

1	Purpose of the Analyzer Script	7
2	Overall Analyzer Data Flow	8
3	Imported Python Packages	8
3.1	shutil	8
3.1.1	Package Origin	8
3.1.2	Purpose	8
3.1.3	Important Function Used	8
3.2	Path from pathlib	9
3.2.1	Package Origin	9
3.2.2	Purpose	9
3.2.3	Important Methods Used	9
3.3	datetime	9
3.3.1	Package Origin	9
3.3.2	Purpose	10
3.3.3	Important Function Used	10
3.4	pandas as pd	10
3.4.1	Package Origin	10
3.4.2	Purpose	10
3.4.3	Important Functions Used	10
3.5	numpy as np	10
3.5.1	Package Origin	11
3.5.2	Purpose	11
3.5.3	Important Functions Used	11
3.6	matplotlib.pyplot as plt	12
3.6.1	Package Origin	12
3.6.2	Purpose	12
3.6.3	Important Functions Used	12

4	Analyzer Settings and Constants	13
4.1	Raw and Processed Data Directories	13
4.1.1	Purpose	13
4.2	File Pattern	13
4.2.1	Purpose	13
4.3	Sensor Names and Axis Names	14
4.3.1	Purpose	14
4.4	Axis Labels	14
4.4.1	Purpose	14
4.5	Reference and Response Sensors	14
4.5.1	Purpose	15
4.6	Transmissibility Epsilon	15
4.6.1	Purpose	15
4.7	Top Peak Settings	15
4.7.1	Purpose	15
4.8	Frequency Bands	15
4.8.1	Purpose	16
5	Processed Directory Creation	16
5.1	Function Origin	16
5.2	Purpose	16
5.3	Inputs	16
5.4	Output	16
6	Raw CSV File Selection	17
6.1	Searching for Raw CSV Files	17
6.1.1	Functions Used	17
6.2	No File Error	17
6.2.1	Purpose	17
6.2.2	Function Origin	17
6.3	User File Selection	17
6.3.1	Functions Used	18
6.4	Latest File Selection	18
6.4.1	Functions Used	18
7	Processed Run Folder Creation	18
7.1	Purpose	19
7.2	Functions and Attributes Used	19
7.3	Duplicate Folder Handling	19
7.4	Folder Creation	19
8	Loading the CSV Data	19
8.1	Function Origin	19
8.2	Purpose	20
8.3	Input	20

8.4	Output	20
9	Data Mode Detection	20
9.1	Three-Sensor Column List	20
9.2	Legacy Columns	20
9.3	Column Check	21
9.3.1	Functions Used	21
9.4	Error Handling	21
10	Raw File Copy for Traceability	21
10.1	Purpose	21
10.2	Function Origin	21
11	Time and Sampling Information	22
11.1	Time Vector Creation	22
11.1.1	Purpose	22
11.1.2	Functions Used	22
11.2	Sampling Interval Calculation	22
11.2.1	Purpose	22
11.2.2	Functions Used	23
11.2.3	Sampling Frequency Equation	23
11.3	Duration and Nyquist Frequency	23
11.3.1	Duration	23
11.3.2	Number of Samples	23
11.3.3	Nyquist Frequency	23
12	Sampling Statistics	24
12.1	Functions Used	24
12.2	Dropped Sample Estimate	24
12.2.1	Purpose	25
13	Sampling Jitter Plot	25
13.1	Purpose	25
13.2	Functions Used	26
14	Sensor Orientation Correction	27
14.1	Purpose	27
15	Custom Analysis Functions	27
15.1	centered_signal()	27
15.1.1	Function Origin	27
15.1.2	Purpose	28
15.1.3	Input	28
15.1.4	Output	28
15.1.5	Internal Library Functions and Methods Used	28
15.1.6	DC Offset Removal Equation	29

15.1.7	Noise-Floor Cleanup	29
15.2	<code>compute_fft()</code>	29
15.2.1	Function Origin	30
15.2.2	Purpose	30
15.2.3	Input	30
15.2.4	Output	30
15.2.5	Internal Library Functions Used	30
15.2.6	Windowing	32
15.2.7	FFT Magnitude	32
15.2.8	Frequency Resolution	32
15.3	<code>find_top_peaks()</code>	33
15.3.1	Function Origin	33
15.3.2	Purpose	34
15.3.3	Inputs	34
15.3.4	Output	34
15.3.5	Internal Built-in Functions Used	34
15.3.6	Local Peak Condition	35
15.3.7	Minimum Spacing Condition	36
15.4	<code>summarize_frequency_bands()</code>	36
15.4.1	Function Origin	36
15.4.2	Purpose	36
15.4.3	Inputs	37
15.4.4	Output	37
15.4.5	Internal Library Functions Used	37
15.4.6	Total Energy	38
15.4.7	Band Energy	38
15.4.8	Energy Percentage	38
15.4.9	RMS-Like Band Value	38
15.5	<code>analyze_column()</code>	38
15.5.1	Function Origin	39
15.5.2	Purpose	39
15.5.3	Input	39
15.5.4	Output	39
15.5.5	Internal Custom Functions Used	40
15.5.6	Internal Library Functions Used	40
15.5.7	RMS Acceleration	41
15.5.8	Peak Acceleration	41
15.5.9	Peak-to-Peak Acceleration	42
15.5.10	Dominant Frequency	42
15.5.11	Time-Domain Plot	42
15.5.12	FFT Plot	42
16	Running the Analysis for Each Sensor-Axis	43
16.1	Purpose	43
16.2	Outputs of This Loop	43

17 Saving Peak and Band Summary CSV Files	43
17.1 Functions Used	44
17.2 Saved Files	44
18 Combined FFT CSV	44
18.1 Purpose	44
18.2 Output File	45
19 Combined Time-Domain CSV	45
19.1 Purpose	45
19.2 Output File	45
20 Acceleration Transmissibility Analysis	45
20.1 Purpose	45
20.2 Condition for Running Transmissibility	46
20.3 Transmissibility Calculation	46
20.3.1 Functions Used	46
20.3.2 Equation	46
20.4 Maximum Transmissibility	46
20.5 Mean and Median Transmissibility	46
20.5.1 Functions Used	47
20.6 Transmissibility Plots	47
20.7 Saved Transmissibility Files	47
21 Combined Time-Domain Plots	48
21.1 Three-Sensor Mode	48
21.2 Saved Files	48
22 Combined FFT Plots	48
22.1 Overlapped FFT by Axis	48
22.2 Saved Files	48
22.3 FFT Plot by Sensor	49
23 Summary Text File	49
23.1 Purpose	49
23.2 Functions Used	50
24 Final Terminal Output	50
25 FFT Theory Summary	50
25.1 Time Domain	50
25.2 Frequency Domain	51
25.3 FFT Interpretation	51
25.4 Dominant Frequency	51
25.5 Nyquist Limit	51

26 Complete Analyzer Execution Flow	51
27 Summary	53

1 Purpose of the Analyzer Script

The purpose of the Python analyzer script, `analyze_fft.py`, is to process the CSV file created by the ESP32 logger and perform vibration analysis.

The analyzer performs both time-domain and frequency-domain analysis. It loads the raw CSV file, estimates the real sampling rate, corrects sensor orientation, removes DC offsets, calculates FFT spectra, detects dominant frequencies, summarizes vibration energy by frequency bands, calculates acceleration transmissibility, and saves plots and CSV files into a processed results folder.

The analyzer acts as the final stage of the project:

CSV File → Signal Processing → FFT Analysis → Plots, Tables, and Summary Files

The main tasks of the analyzer are:

1. Locate available raw CSV files.
2. Allow the user to select which file to analyze.
3. Create a processed output folder.
4. Load the selected CSV file using Pandas.
5. Determine whether the data is three-sensor data or legacy single-sensor data.
6. Copy the raw CSV into the processed folder for traceability.
7. Estimate the real sampling rate from the timestamp column.
8. Calculate sampling quality statistics.
9. Plot sampling jitter.
10. Apply sensor orientation correction.
11. Remove DC offset from each signal.
12. Apply a small noise-floor cleanup.
13. Compute FFT spectra for each sensor axis.
14. Identify dominant frequencies and top FFT peaks.
15. Summarize frequency-band vibration energy.
16. Save time-domain and frequency-domain CSV files.
17. Generate individual and combined plots.
18. Calculate acceleration transmissibility using sensor 3 as the reference.
19. Write a final text summary file.

2 Overall Analyzer Data Flow

The analyzer follows this data flow:

`pd.read_csv()` → Time Vector → Sampling Rate Estimate → Centered Signals → FFT → Peak Detection

The main signal-processing path for one sensor-axis is:

Raw Acceleration Column → Orientation Correction → Mean Removal → Noise Floor Cleanup → Hanning

3 Imported Python Packages

The analyzer imports the following packages:

```
import shutil
from pathlib import Path
from datetime import datetime

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

3.1 `shutil`

3.1.1 Package Origin

`shutil` is a standard Python library.

3.1.2 Purpose

The `shutil` package is used for high-level file operations.

In this script, it is used to copy the raw CSV file into the processed output folder.

3.1.3 Important Function Used

```
shutil.copy2(CSV_FILE, raw_copy_path)
```

Function Origin `copy2()` comes from Python's standard `shutil` module.

Purpose Copies a file while preserving metadata when possible.

Inputs

- Source file path
- Destination file path

Output The copied file is created in the processed folder.

Why It Is Used This preserves a copy of the raw data used for the analysis. This is useful for traceability, because every processed folder contains the original raw input file.

3.2 Path from pathlib

```
from pathlib import Path
```

3.2.1 Package Origin

pathlib is a standard Python library.

3.2.2 Purpose

Path is used to manage file and folder paths.

It allows paths to be handled as objects instead of plain text strings.

3.2.3 Important Methods Used

- Path(...)
- mkdir()
- glob()
- exists()
- stat()
- stem
- name

3.3 datetime

```
from datetime import datetime
```

3.3.1 Package Origin

datetime is a standard Python library.

3.3.2 Purpose

The analyzer uses `datetime` to create a timestamped processed folder name if a folder with the same name already exists.

3.3.3 Important Function Used

```
datetime.now().strftime("%Y-%m-%d_%H-%M-%S")
```

This creates a timestamp string such as:

2026-05-12_09-20-15

3.4 pandas as pd

```
import pandas as pd
```

3.4.1 Package Origin

`pandas` is an external Python data-analysis library.

3.4.2 Purpose

Pandas is used to read, organize, manipulate, and save tabular data.

In this script, Pandas is used to:

- Read the raw CSV file.
- Store data in a `DataFrame`.
- Create summary tables.
- Save processed CSV files.

3.4.3 Important Functions Used

- `pd.read_csv()`
- `pd.DataFrame()`
- `DataFrame.to_csv()`

3.5 numpy as np

```
import numpy as np
```

3.5.1 Package Origin

numpy is an external Python numerical-computing library.

3.5.2 Purpose

NumPy is used for numerical operations and signal processing.

In this script, NumPy is used to:

- Convert DataFrame columns into arrays.
- Calculate time-step differences.
- Estimate sampling rate.
- Calculate mean, minimum, maximum, and standard deviation.
- Remove signal mean.
- Apply a Hanning window.
- Compute FFT spectra.
- Generate frequency bins.
- Calculate RMS, peak, and peak-to-peak values.
- Detect peaks.
- Calculate frequency-band energy.
- Calculate transmissibility ratios.

3.5.3 Important Functions Used

- `np.diff()`
- `np.mean()`
- `np.min()`
- `np.max()`
- `np.std()`
- `np.sqrt()`
- `np.abs()`
- `np.hanning()`
- `np.fft.rfft()`

- `np.fft.rfftfreq()`
- `np.argmax()`
- `np.sum()`
- `np.any()`
- `np.maximum()`
- `np.median()`
- `np.arange()`

3.6 matplotlib.pyplot as plt

```
import matplotlib.pyplot as plt
```

3.6.1 Package Origin

matplotlib is an external Python plotting library.

3.6.2 Purpose

matplotlib.pyplot is used to generate and save graphs.

In this script, it creates:

- Sampling jitter plots.
- Acceleration versus time plots.
- FFT magnitude plots.
- Overlapped FFT plots.
- Transmissibility plots.

3.6.3 Important Functions Used

- `plt.figure()`
- `plt.plot()`
- `plt.xlabel()`
- `plt.ylabel()`
- `plt.title()`
- `plt.grid()`

- plt.legend()
- plt.xlim()
- plt.xticks()
- plt.savefig()
- plt.show()

4 Analyzer Settings and Constants

4.1 Raw and Processed Data Directories

```
RAW_DATA_DIR = Path(r"F:\Rlenovo99\Documents\Projects\Structural_Vibration_Analyzer\
Data\raw")
PROCESSED_DIR = Path(r"F:\Rlenovo99\Documents\Projects\
Structural_Vibration_Analyzer\Data\processed")
```

4.1.1 Purpose

These paths define where the script looks for raw CSV data and where it saves processed results.

RAW_DATA_DIR

contains files generated by the logger.

PROCESSED_DIR

contains folders generated by the analyzer.

4.2 File Pattern

```
FILE_PATTERN = "esp32_vibration_*.csv"
```

4.2.1 Purpose

This defines which raw CSV files the analyzer will search for.

The analyzer only looks for files matching:

esp32_vibration_*.csv

This matches the logger output filename format.

4.3 Sensor Names and Axis Names

```
NUM_SENSORS = 3
SENSOR_NAMES = [f"s{i}" for i in range(1, NUM_SENSORS + 1)]
AXIS_SUFFIXES = ["ax_g", "ay_g", "az_g"]
```

4.3.1 Purpose

The analyzer expects three sensors:

$$s1, s2, s3$$

and three acceleration axes:

$$a_x, a_y, a_z$$

The corresponding CSV columns are:

$$s1_ax_g, s1_ay_g, s1_az_g$$
$$s2_ax_g, s2_ay_g, s2_az_g$$
$$s3_ax_g, s3_ay_g, s3_az_g$$

4.4 Axis Labels

```
AXIS_LABELS = {
    "ax_g": "X_Axis",
    "ay_g": "Y_Axis",
    "az_g": "Z_Axis",
}
```

4.4.1 Purpose

This dictionary provides readable axis names for plot titles.

For example:

$$\text{"ax_g"} \rightarrow \text{"X Axis"}$$

4.5 Reference and Response Sensors

```
REFERENCE_SENSOR = "s3"
RESPONSE_SENSORS = ["s1", "s2"]
```

4.5.1 Purpose

Sensor 3 is treated as the reference or input sensor.

Sensors 1 and 2 are treated as response sensors.

This is used during acceleration transmissibility analysis.

The transmissibility ratio is calculated as:

$$T(f) = \frac{A_{\text{response}}(f)}{A_{\text{reference}}(f)}$$

4.6 Transmissibility Epsilon

```
TRANSMISSIBILITY_EPSILON = 1e-12
```

4.6.1 Purpose

This small value prevents division by zero during transmissibility calculations.

The denominator is protected using:

$$\max(A_{\text{reference}}, 10^{-12})$$

4.7 Top Peak Settings

```
NUM_TOP_PEAKE = 10  
PEAK_MIN_SPACING_HZ = 2.0
```

4.7.1 Purpose

The analyzer reports the top 10 FFT peaks for each sensor-axis.

The minimum spacing between selected peaks is:

2.0 Hz

This avoids reporting several nearby bins from the same broad vibration peak.

4.8 Frequency Bands

```
FREQUENCY_BANDS = [  
    (0, 10),  
    (10, 25),  
    (25, 50),  
    (50, 100),  
    (100, 200),  
    (200, None),  
]
```

4.8.1 Purpose

These bands are used to summarize vibration energy in different frequency ranges.

The bands are:

0 – 10 Hz

10 – 25 Hz

25 – 50 Hz

50 – 100 Hz

100 – 200 Hz

200 + Hz

5 Processed Directory Creation

`PROCESSED_DIR.mkdir(parents=True, exist_ok=True)`

5.1 Function Origin

`mkdir()` is a method from Python's `pathlib.Path` object.

5.2 Purpose

This creates the processed results folder if it does not already exist.

5.3 Inputs

- `parents=True`: creates missing parent folders.
- `exist_ok=True`: does not raise an error if the folder already exists.

5.4 Output

No returned data is used.

The folder is created on disk.

6 Raw CSV File Selection

6.1 Searching for Raw CSV Files

```
csv_files = sorted(RAW_DATA_DIR.glob(FILE_PATTERN))
```

6.1.1 Functions Used

`RAW_DATA_DIR.glob()` Origin:

`pathlib.Path`

Purpose:

Finds files matching the pattern:

`esp32_vibration_*.csv`

`sorted()` Origin:

Python built-in function.

Purpose:

Sorts the list of matching CSV files.

6.2 No File Error

```
if not csv_files:  
    raise FileNotFoundError(...)
```

6.2.1 Purpose

If no raw CSV files are found, the script stops and raises an error.

6.2.2 Function Origin

`FileNotFoundError` is a built-in Python exception.

6.3 User File Selection

The analyzer prints all available files and lets the user select one.

```
choice = input(  
    "\nEnter the file number to analyze, or press ENTER to use the latest file: "  
) .strip()
```

6.3.1 Functions Used

`input()` Origin:

Python built-in function.

Purpose:

Reads user input from the terminal.

`strip()` Origin:

Python string method.

Purpose:

Removes extra spaces and newline characters.

6.4 Latest File Selection

If the user presses ENTER:

```
CSV_FILE = max(csv_files, key=lambda p: p.stat().st_mtime)
```

6.4.1 Functions Used

`max()` Origin:

Python built-in function.

Purpose:

Finds the file with the latest modification time.

`p.stat().st_mtime` Origin:

`pathlib.Path`

Purpose:

Returns the file modification time.

`lambda` Origin:

Python language feature.

Purpose:

Defines a small temporary function used as the key for selecting the latest file.

7 Processed Run Folder Creation

```
raw_stem = CSV_FILE.stem  
run_folder = PROCESSED_DIR / f"{raw_stem}_processed_V2"
```

7.1 Purpose

The analyzer creates a new folder for each processed data run.

The processed folder name is based on the raw CSV filename.

7.2 Functions and Attributes Used

`CSV_FILE.stem` Origin:

`pathlib.Path`

Purpose:

Gets the filename without the file extension.

Path Join Operator / Origin:

`pathlib.Path`

Purpose:

Combines the processed folder path with the new run folder name.

7.3 Duplicate Folder Handling

```
if run_folder.exists():
    timestamp = datetime.now().strftime("%Y-%m-%d_%H-%M-%S")
    run_folder = PROCESSED_DIR / f"{raw_stem}_processed_V2_{timestamp}"
```

If the processed folder already exists, the analyzer appends a timestamp to avoid overwriting previous results.

7.4 Folder Creation

```
run_folder.mkdir(parents=True, exist_ok=True)
```

This creates the processed run folder.

8 Loading the CSV Data

```
df = pd.read_csv(CSV_FILE)
```

8.1 Function Origin

`pd.read_csv()` comes from the Pandas library.

8.2 Purpose

This reads the raw CSV file into a Pandas DataFrame.

8.3 Input

- `CSV_FILE`: path to the selected raw CSV file.

8.4 Output

Returns a Pandas DataFrame named:

`df`

The DataFrame stores the CSV data in rows and columns.

9 Data Mode Detection

The analyzer supports two data modes:

1. Three-sensor mode
2. Single-sensor legacy mode

9.1 Three-Sensor Column List

```
multi_sensor_columns = [  
    f"{sensor}_{axis}"  
    for sensor in SENSOR_NAMES  
    for axis in AXIS_SUFFIXES  
]
```

This creates the expected columns:

s1_ax_g, s1_ay_g, s1_az_g

s2_ax_g, s2_ay_g, s2_az_g

s3_ax_g, s3_ay_g, s3_az_g

9.2 Legacy Columns

```
legacy_columns = ["ax_g", "ay_g", "az_g"]
```

These are used for older single-sensor CSV files.

9.3 Column Check

```
if all(column in df.columns for column in multi_sensor_columns):  
    data_mode = "three_sensor"
```

9.3.1 Functions Used

`all()` Origin:

Python built-in function.

Purpose:

Returns True only if all required columns exist.

`df.columns` Origin:

Pandas DataFrame attribute.

Purpose:

Contains the column names in the CSV file.

9.4 Error Handling

If neither three-sensor nor legacy columns are found, the script raises:

```
raise ValueError(...)
```

`ValueError` is a built-in Python exception.

10 Raw File Copy for Traceability

```
raw_copy_path = run_folder / "raw_data_copy.csv"  
shutil.copy2(CSV_FILE, raw_copy_path)
```

10.1 Purpose

This copies the original CSV file into the processed folder.

This makes the processed results folder self-contained.

10.2 Function Origin

`shutil.copy2()` comes from Python's standard `shutil` module.

11 Time and Sampling Information

11.1 Time Vector Creation

```
t = df["t_us"].to_numpy() / 1_000_000.0  
t = t - t[0]
```

11.1.1 Purpose

The CSV stores time in microseconds.

The analyzer converts time into seconds:

$$t_s = \frac{t_{\mu s}}{1,000,000}$$

Then it shifts the time vector so that the first sample is at:

$$t = 0$$

11.1.2 Functions Used

`df["t_us"]` Origin:

Pandas DataFrame column selection.

Purpose:

Selects the timestamp column.

`to_numpy()` Origin:

Pandas Series method.

Purpose:

Converts the column into a NumPy array.

11.2 Sampling Interval Calculation

```
dt_values = np.diff(t)  
dt = np.mean(dt_values)  
fs = 1 / dt
```

11.2.1 Purpose

The analyzer calculates the actual sampling interval and sampling frequency from the recorded timestamps.

11.2.2 Functions Used

`np.diff()` Origin:

NumPy

Purpose:

Calculates differences between consecutive values.

$$\Delta t_i = t_{i+1} - t_i$$

`np.mean()` Origin:

NumPy

Purpose:

Calculates the average sampling interval.

$$\overline{\Delta t} = \frac{1}{N-1} \sum_{i=1}^{N-1} \Delta t_i$$

11.2.3 Sampling Frequency Equation

The estimated sampling frequency is:

$$f_s = \frac{1}{\overline{\Delta t}}$$

11.3 Duration and Nyquist Frequency

```
duration = t[-1] - t[0]
num_samples = len(t)
nyquist = fs / 2
```

11.3.1 Duration

$$T_{\text{duration}} = t_{\text{last}} - t_{\text{first}}$$

11.3.2 Number of Samples

$$N = \text{len}(t)$$

11.3.3 Nyquist Frequency

$$f_N = \frac{f_s}{2}$$

The Nyquist frequency is the highest frequency that can be represented without aliasing.

12 Sampling Statistics

The script creates a dictionary called:

`sampling_stats`

It stores:

- Mean sampling interval
- Minimum sampling interval
- Maximum sampling interval
- Standard deviation of sampling interval
- Estimated sampling frequency
- Nyquist frequency
- Duration
- Number of samples

12.1 Functions Used

`np.mean()` Calculates average time step.

`np.min()` Calculates smallest time step.

`np.max()` Calculates largest time step.

`np.std()` Calculates standard deviation of the time step.

12.2 Dropped Sample Estimate

If the CSV contains `sample_index`, the analyzer calculates:

```
sample_index_diff = np.diff(sample_index)
dropped_samples = int(np.sum(sample_index_diff - 1))
repeated_or_reversed = int(np.sum(sample_index_diff <= 0))
```


12.2.1 Purpose

The sample index should increase by one each frame.

Ideally:

$$\Delta n = 1$$

If:

$$\Delta n > 1$$

then samples may have been skipped.

The estimated dropped samples are:

$$\sum (\Delta n - 1)$$

The analyzer also checks for repeated or reversed indices:

$$\Delta n \leq 0$$

13 Sampling Jitter Plot

```
plt.figure(figsize=(12, 5))
plt.plot(t[1:], dt_values)
plt.xlabel("Time [s]")
plt.ylabel("Sampling_interval_dt [s]")
plt.title("Sampling_Jitter_/Time_Step_Variation")
plt.grid(True)
plt.savefig(sampling_jitter_plot_path, dpi=300, bbox_inches="tight")
plt.show()
```

13.1 Purpose

This plot shows how the sampling interval changes over time.

Ideally, the sampling interval should be nearly constant.

If the target sampling rate is:

$$800 \text{ Hz}$$

then the ideal sampling interval is:

$$\Delta t = \frac{1}{800} = 0.00125 \text{ s}$$

13.2 Functions Used

`plt.figure()` Origin:

`matplotlib.pyplot`

Purpose:

Creates a new figure.

`plt.plot()` Origin:

`matplotlib.pyplot`

Purpose:

Plots sampling interval versus time.

`plt.xlabel()`, `plt.ylabel()`, `plt.title()` Origin:

`matplotlib.pyplot`

Purpose:

Adds labels and title.

`plt.grid()` Origin:

`matplotlib.pyplot`

Purpose:

Displays grid lines.

`plt.savefig()` Origin:

`matplotlib.pyplot`

Purpose:

Saves the plot as an image file.

`plt.show()` Origin:

`matplotlib.pyplot`

Purpose:

Displays the plot.

14 Sensor Orientation Correction

```
SENSOR_AXIS_SIGN = {  
    "s1": {"ax_g": 1, "ay_g": 1, "az_g": 1},  
    "s2": {"ax_g": -1, "ay_g": 1, "az_g": -1},  
    "s3": {"ax_g": 1, "ay_g": 1, "az_g": 1},  
}
```

14.1 Purpose

This dictionary corrects sensor orientation differences.

Sensor 2 is mounted in a mirrored orientation, so its x and z axes are inverted.

Therefore:

$$s2_{x,\text{corrected}} = -s2_x$$

$$s2_{z,\text{corrected}} = -s2_z$$

The y-axis remains unchanged.

15 Custom Analysis Functions

15.1 centered_signal()

```
def centered_signal(column_name):  
    signal_raw = df[column_name].to_numpy()  
  
    if "_" in column_name:  
        sensor, axis = column_name.split("_", 1)  
  
        sign = SENSOR_AXIS_SIGN.get(sensor, {}).get(axis, 1)  
  
        signal_raw = sign * signal_raw  
  
    signal_centered = signal_raw - np.mean(signal_raw)  
  
    signal_centered[np.abs(signal_centered) < 0.002] = 0  
  
    return signal_centered
```

15.1.1 Function Origin

This is a custom function written specifically for this project.

It does not come from NumPy, Pandas, or Matplotlib.

15.1.2 Purpose

This function prepares one acceleration column for analysis.

It performs:

1. Column extraction.
2. Sensor orientation correction.
3. DC offset removal.
4. Small noise-floor cleanup.

15.1.3 Input

- `column_name`: name of the acceleration column to process.

Example:

`s2_ax_g`

15.1.4 Output

Returns a NumPy array containing the corrected and centered signal.

x_{centered}

15.1.5 Internal Library Functions and Methods Used

`df[column_name]` Origin:

Pandas DataFrame column selection.

Purpose:

Extracts the requested acceleration column.

`to_numpy()` Origin:

Pandas Series method.

Purpose:

Converts the column into a NumPy array.

`split()` Origin:

Python string method.

Purpose:

Splits the column name into sensor name and axis name.

Example:

`s2_ax_g` $\rightarrow$ `s2, ax_g`

`dict.get()` Origin:
Python dictionary method.
Purpose:
Safely retrieves the sign correction factor.

`np.mean()` Origin:

NumPy

Purpose:
Calculates the average value of the signal.

`np.abs()` Origin:

NumPy

Purpose:
Calculates absolute values for noise-floor cleanup.

15.1.6 DC Offset Removal Equation

The signal mean is:

$$\bar{x} = \frac{1}{N} \sum_{i=1}^N x_i$$

The centered signal is:

$$x_{\text{centered},i} = x_i - \bar{x}$$

15.1.7 Noise-Floor Cleanup

The code sets very small values to zero:

$$|x| < 0.002g \Rightarrow x = 0$$

This reduces very small residual noise in the centered signal.

15.2 compute_fft()

```
def compute_fft(signal_centered):  
    window = np.hanning(len(signal_centered))  
    signal_windowed = signal_centered * window  
  
    fft_values = np.fft.rfft(signal_windowed)  
    freqs = np.fft.rfftfreq(len(signal_windowed), d=dt)
```

```
magnitude = np.abs(fft_values) * 2 / len(signal_windowed)

return freqs, magnitude
```

15.2.1 Function Origin

This is a custom function written specifically for this project.

The function itself does not come from a library.

However, it uses several NumPy functions internally.

15.2.2 Purpose

This function calculates the FFT frequency spectrum of one centered acceleration signal.

15.2.3 Input

- **signal_centered**: acceleration signal after orientation correction and DC offset removal.

15.2.4 Output

Returns two NumPy arrays:

- **freqs**: frequency values in Hz.
- **magnitude**: FFT magnitude values in g .

15.2.5 Internal Library Functions Used

`np.hanning()` Origin:

NumPy

Specifically:

`numpy.hanning()`

Purpose:

Creates a Hanning window.

Input:

N = number of samples

Output:

A window vector of length N .

The Hanning window reduces spectral leakage.

`len()` Origin:

Python built-in function.

Purpose:

Gets the number of samples in the signal.

`np.fft.rfft()` Origin:

`numpy.fft`

Specifically:

`numpy.fft.rfft()`

Purpose:

Computes the Fast Fourier Transform of a real-valued signal.

Input:

$x(t)$

Output:

$X(f)$

The output is complex-valued.

`np.fft.rfftfreq()` Origin:

`numpy.fft`

Specifically:

`numpy.fft.rfftfreq()`

Purpose:

Generates the frequency values corresponding to the real FFT bins.

Inputs:

- Number of samples
- Sampling interval $d=dt$

Output:

Frequency vector in Hz.

`np.abs()` Origin:

NumPy

Purpose:

Calculates the magnitude of the complex FFT values.

15.2.6 Windowing

The signal is multiplied by a Hanning window:

$$x_w[n] = x[n]w[n]$$

The Hanning window is approximately:

$$w[n] = 0.5 \left(1 - \cos \left(\frac{2\pi n}{N-1} \right) \right)$$

Windowing reduces spectral leakage caused by discontinuities at the start and end of the sampled signal.

15.2.7 FFT Magnitude

The FFT output is complex:

$$X(f) = a + jb$$

The magnitude is:

$$|X(f)| = \sqrt{a^2 + b^2}$$

The script calculates:

$$M(f) = \frac{2}{N} |X(f)|$$

The factor of 2 is used because `rfft()` returns only the positive-frequency half of the spectrum.

15.2.8 Frequency Resolution

The FFT frequency spacing is:

$$\Delta f = \frac{f_s}{N}$$

where:

- f_s is the estimated sampling rate.
- N is the number of samples.

15.3 find_top_peaks()

```
def find_top_peaks(freqs, magnitude, num_peaks=NUM_TOP_PEAKEs, min_spacing_hz=
    PEAK_MIN_SPACING_HZ):
    if len(freqs) < 3:
        return []

    candidate_indices = []

    for i in range(1, len(magnitude) - 1):
        if magnitude[i] > magnitude[i - 1] and magnitude[i] > magnitude[i + 1]:
            candidate_indices.append(i)

    candidate_indices = sorted(candidate_indices, key=lambda i: magnitude[i],
        reverse=True)

    selected_indices = []

    for idx in candidate_indices:
        freq = freqs[idx]

        if freq <= 0:
            continue

        too_close = any(abs(freq - freqs[selected]) < min_spacing_hz for selected
            in selected_indices)

        if not too_close:
            selected_indices.append(idx)

        if len(selected_indices) >= num_peaks:
            break

    peaks = []

    for rank, idx in enumerate(selected_indices, start=1):
        peaks.append({
            "rank": rank,
            "frequency_hz": float(freqs[idx]),
            "magnitude_g": float(magnitude[idx]),
        })

    return peaks
```

15.3.1 Function Origin

This is a custom function written specifically for this project.

It does not come from SciPy or NumPy.

15.3.2 Purpose

This function identifies the strongest local FFT peaks without requiring the SciPy library.

15.3.3 Inputs

- `freqs`: frequency vector.
- `magnitude`: FFT magnitude vector.
- `num_peaks`: number of peaks to report.
- `min_spacing_hz`: minimum allowed spacing between selected peaks.

15.3.4 Output

Returns a list of dictionaries.

Each dictionary contains:

- Peak rank
- Peak frequency in Hz
- Peak magnitude in g

15.3.5 Internal Built-in Functions Used

`len()` Origin:

Python built-in function.

Purpose:

Checks array length and number of selected peaks.

`range()` Origin:

Python built-in function.

Purpose:

Creates the index range for checking local peaks.

`sorted()` Origin:

Python built-in function.

Purpose:

Sorts peak candidates from largest to smallest magnitude.

`lambda` Origin:
Python language feature.
Purpose:
Defines the sorting key:

`magnitude[i]`

`any()` Origin:
Python built-in function.
Purpose:
Checks whether the current peak is too close to an already selected peak.

`abs()` Origin:
Python built-in function.
Purpose:
Calculates absolute frequency spacing.

`enumerate()` Origin:
Python built-in function.
Purpose:
Creates rank numbers for selected peaks.

`float()` Origin:
Python built-in function.
Purpose:
Converts NumPy numeric values into standard Python floats for saving.

15.3.6 Local Peak Condition

A point is considered a local peak if:

$$M_i > M_{i-1}$$

and:

$$M_i > M_{i+1}$$

where:

$$M_i$$

is the FFT magnitude at frequency bin i .

15.3.7 Minimum Spacing Condition

A new peak is accepted only if:

$$|f_{\text{new}} - f_{\text{selected}}| \geq \text{min_spacing_hz}$$

for all previously selected peaks.

This avoids selecting multiple adjacent bins from the same broad peak.

15.4 summarize_frequency_bands()

```
def summarize_frequency_bands(freqs, magnitude, column_name):
    band_rows = []

    total_energy = np.sum(magnitude ** 2)

    for low, high in FREQUENCY_BANDS:
        if high is None:
            mask = freqs >= low
            band_label = f"{low}+_Hz"
        else:
            mask = (freqs >= low) & (freqs < high)
            band_label = f"{low}-{high}_Hz"

        band_energy = np.sum(magnitude[mask] ** 2)
        band_rms_like = np.sqrt(np.mean(magnitude[mask] ** 2)) if np.any(mask) else 0
        energy_percent = (band_energy / total_energy * 100) if total_energy > 0 else 0

        band_rows.append({
            "column": column_name,
            "band": band_label,
            "band_energy_g2": float(band_energy),
            "band_rms_like_g": float(band_rms_like),
            "energy_percent": float(energy_percent),
        })

    return band_rows
```

15.4.1 Function Origin

This is a custom function written specifically for this project.

15.4.2 Purpose

This function summarizes how much FFT vibration energy is contained in each frequency band.

15.4.3 Inputs

- `freqs`: frequency vector.
- `magnitude`: FFT magnitude vector.
- `column_name`: name of the sensor-axis being analyzed.

15.4.4 Output

Returns a list of dictionaries.

Each dictionary contains:

- Column name
- Frequency band label
- Band energy
- RMS-like band value
- Energy percentage

15.4.5 Internal Library Functions Used

`np.sum()` Origin:

NumPy

Purpose:

Calculates total and band energy.

`np.sqrt()` Origin:

NumPy

Purpose:

Calculates RMS-like values.

`np.mean()` Origin:

NumPy

Purpose:

Calculates average squared magnitude inside a band.

`np.any()` Origin:

NumPy

Purpose:

Checks whether a frequency mask contains any values.

`float()` Origin:

Python built-in function.

Purpose:

Converts values into normal Python floats.

15.4.6 Total Energy

The total spectral energy approximation is:

$$E_{\text{total}} = \sum M(f)^2$$

15.4.7 Band Energy

For a frequency band:

$$f_{\text{low}} \leq f < f_{\text{high}}$$

the band energy is:

$$E_{\text{band}} = \sum_{f \in \text{band}} M(f)^2$$

15.4.8 Energy Percentage

$$\%E_{\text{band}} = \frac{E_{\text{band}}}{E_{\text{total}}} \times 100$$

15.4.9 RMS-Like Band Value

$$M_{\text{RMS-like}} = \sqrt{\frac{1}{N_{\text{band}}} \sum_{f \in \text{band}} M(f)^2}$$

15.5 analyze_column()

```
def analyze_column(column_name):  
    signal = centered_signal(column_name)  
  
    rms_g = np.sqrt(np.mean(signal ** 2))  
    peak_g = np.max(np.abs(signal))  
    peak_to_peak_g = np.max(signal) - np.min(signal)
```

```
freqs, magnitude = compute_fft(signal)

peak_index = np.argmax(magnitude[1:]) + 1
dominant_freq = freqs[peak_index]
dominant_mag = magnitude[peak_index]

top_peaks = find_top_peaks(freqs, magnitude)
band_summary = summarize_frequency_bands(freqs, magnitude, column_name)

...
```

15.5.1 Function Origin

This is a custom function written specifically for this project.

15.5.2 Purpose

This function performs a complete analysis for one sensor-axis column.

For example:

s1_ax_g

or:

s3_az_g

15.5.3 Input

- `column_name`: the name of the acceleration column to analyze.

15.5.4 Output

Returns a dictionary containing:

- Column name
- RMS acceleration
- Peak acceleration
- Peak-to-peak acceleration
- Dominant frequency
- Dominant magnitude
- FFT CSV filename

- Time-domain plot filename
- FFT plot filename
- Top FFT peaks
- Frequency band summary

15.5.5 Internal Custom Functions Used

`centered_signal()` Prepares the signal by correcting orientation, removing the mean, and applying noise cleanup.

`compute_fft()` Calculates the frequency spectrum.

`find_top_peaks()` Finds the strongest local FFT peaks.

`summarize_frequency_bands()` Summarizes spectral energy by frequency band.

15.5.6 Internal Library Functions Used

`np.sqrt()` Origin:

NumPy

Purpose:

Calculates square roots for RMS.

`np.mean()` Origin:

NumPy

Purpose:

Calculates the mean of squared signal values.

`np.max()` Origin:

NumPy

Purpose:

Calculates peak and maximum signal values.

`np.min()` Origin:

NumPy

Purpose:

Calculates minimum signal values.

`np.abs()` Origin:

NumPy

Purpose:

Calculates absolute values for peak acceleration.

`np.argmax()` Origin:

NumPy

Purpose:

Finds the index of the largest FFT magnitude.

`pd.DataFrame()` Origin:

Pandas

Purpose:

Creates a table containing the FFT results.

`to_csv()` Origin:

Pandas DataFrame method.

Purpose:

Saves FFT results to a CSV file.

`plt.figure()`, `plt.plot()`, `plt.savefig()`, `plt.show()` Origin:

matplotlib.pyplot

Purpose:

Creates, saves, and displays time-domain and FFT plots.

15.5.7 RMS Acceleration

The RMS acceleration is calculated as:

$$a_{\text{RMS}} = \sqrt{\frac{1}{N} \sum_{i=1}^N a_i^2}$$

This gives an overall measure of vibration intensity.

15.5.8 Peak Acceleration

The peak acceleration is:

$$a_{\text{peak}} = \max(|a_i|)$$

15.5.9 Peak-to-Peak Acceleration

The peak-to-peak acceleration is:

$$a_{\text{p-p}} = a_{\text{max}} - a_{\text{min}}$$

15.5.10 Dominant Frequency

The script finds the dominant frequency using:

$$\text{peak_index} = \arg \max(M(f)), \quad f > 0$$

The DC bin is ignored by using:

$$\text{magnitude}[1:]$$

The corresponding dominant frequency is:

$$f_{\text{dominant}} = f[\text{peak_index}]$$

15.5.11 Time-Domain Plot

The function creates a plot of:

$$a(t)$$

versus:

$$t$$

This shows how acceleration changes with time.

15.5.12 FFT Plot

The function creates a plot of:

$$M(f)$$

versus:

$$f$$

This shows which vibration frequencies are present.

The x-axis is limited to:

$$0 \leq f \leq f_N$$

where:

$$f_N = \frac{f_s}{2}$$

The x-axis tick marks are placed every 25 Hz.

16 Running the Analysis for Each Sensor-Axis

```
summary_results = []
all_peak_rows = []
all_band_rows = []

for column_name in columns_to_analyze:
    print(f"\nAnalyzing: {column_name}")
    result = analyze_column(column_name)
    summary_results.append(result)
```

16.1 Purpose

This loop analyzes every acceleration column.

In three-sensor mode, the analyzed columns are:

s1_ax_g, s1_ay_g, s1_az_g

s2_ax_g, s2_ay_g, s2_az_g

s3_ax_g, s3_ay_g, s3_az_g

So the analyzer processes:

$3 \text{ sensors} \times 3 \text{ axes} = 9 \text{ signals}$

16.2 Outputs of This Loop

The loop builds:

- `summary_results`: general result dictionaries.
- `all_peak_rows`: all top FFT peaks.
- `all_band_rows`: all frequency-band summaries.

17 Saving Peak and Band Summary CSV Files

```
pd.DataFrame(all_peak_rows).to_csv(peak_table_csv_path, index=False)
pd.DataFrame(all_band_rows).to_csv(band_summary_csv_path, index=False)
```

17.1 Functions Used

`pd.DataFrame()` Origin:

Pandas

Purpose:

Converts lists of dictionaries into tables.

`to_csv()` Origin:

Pandas DataFrame method.

Purpose:

Saves the table as a CSV file.

17.2 Saved Files

The analyzer saves:

- `top_fft_peaks_all_sensor_axes.csv`
- `frequency_band_summary_all_sensor_axes.csv`

18 Combined FFT CSV

```
combined_fft_df = None

for column_name in columns_to_analyze:
    signal = centered_signal(column_name)
    freqs, magnitude = compute_fft(signal)

    if combined_fft_df is None:
        combined_fft_df = pd.DataFrame({"Frequency_Hz": freqs})

    combined_fft_df[f"{column_name}_Magnitude_g"] = magnitude
```

18.1 Purpose

This section combines all FFT results into one CSV file.

Instead of saving one separate FFT file per sensor-axis only, this creates a combined table with:

- One frequency column
- One magnitude column for each sensor-axis

18.2 Output File

```
fft_results_all_sensor_axes.csv
```

The script also saves a legacy filename:

```
fft_results_all_axes.csv
```

19 Combined Time-Domain CSV

```
time_domain_data = {"Time_s": t}

for column_name in columns_to_analyze:
    time_domain_data[f"{column_name}_centered"] = centered_signal(column_name)

time_domain_df = pd.DataFrame(time_domain_data)
```

19.1 Purpose

This creates a CSV file containing all centered time-domain acceleration signals.

19.2 Output File

```
time_domain_centered_all_sensor_axes.csv
```

The script also saves a legacy filename:

```
time_domain_centered_all_axes.csv
```

20 Acceleration Transmissibility Analysis

20.1 Purpose

Transmissibility compares the vibration magnitude at one location to the vibration magnitude at another location.

In this script:

- Sensor 3 is the reference/input sensor.
- Sensors 1 and 2 are response sensors.

The transmissibility ratio is:

$$T(f) = \frac{A_{\text{response}}(f)}{A_{\text{reference}}(f)}$$

20.2 Condition for Running Transmissibility

```
if data_mode == "three_sensor" and REFERENCE_SENSOR in SENSOR_NAMES:
```

Transmissibility is only calculated if three-sensor data is detected.

20.3 Transmissibility Calculation

```
transmissibility = response_mag / np.maximum(reference_mag,  
TRANSMISSIBILITY_EPSILON)
```

20.3.1 Functions Used

`np.maximum()` Origin:

NumPy

Purpose:

Compares the reference magnitude to a small epsilon value and uses the larger value.

This prevents division by zero.

20.3.2 Equation

$$T(f) = \frac{A_{\text{response}}(f)}{\max(A_{\text{reference}}(f), 10^{-12})}$$

20.4 Maximum Transmissibility

The script removes the DC bin:

$$f > 0$$

and finds the maximum transmissibility:

$$T_{\max} = \max(T(f))$$

The frequency where it occurs is:

$$f_{T_{\max}}$$

20.5 Mean and Median Transmissibility

The script calculates:

$$T_{\text{mean}} = \frac{1}{N} \sum_{i=1}^N T_i$$

and:

$$T_{\text{median}} = \text{median}(T_i)$$

20.5.1 Functions Used

`np.argmax()` Finds the index of maximum transmissibility.

`np.mean()` Calculates mean transmissibility.

`np.median()` Calculates median transmissibility.

20.6 Transmissibility Plots

The analyzer creates one transmissibility plot per axis:

- x-axis transmissibility
- y-axis transmissibility
- z-axis transmissibility

Each plot compares:

$$s1/s3$$

and:

$$s2/s3$$

20.7 Saved Transmissibility Files

The script saves:

- `transmissibility_results_all_axes.csv`
- `transmissibility_summary.csv`
- `transmissibility_x_axis_reference_s3.png`
- `transmissibility_y_axis_reference_s3.png`
- `transmissibility_z_axis_reference_s3.png`

21 Combined Time-Domain Plots

21.1 Three-Sensor Mode

If three-sensor data is detected, the script creates one plot per sensor with all three axes overlaid.

For each sensor:

$$s1, s2, s3$$

the plot includes:

$$a_x(t), a_y(t), a_z(t)$$

21.2 Saved Files

The saved files are:

- `time_signal_all_axes_s1.png`
- `time_signal_all_axes_s2.png`
- `time_signal_all_axes_s3.png`

22 Combined FFT Plots

22.1 Overlapped FFT by Axis

The analyzer creates one overlapped FFT plot per axis.

For the x-axis plot, it overlays:

$$s1_ax, s2_ax, s3_ax$$

For the y-axis plot, it overlays:

$$s1_ay, s2_ay, s3_ay$$

For the z-axis plot, it overlays:

$$s1_az, s2_az, s3_az$$

22.2 Saved Files

The saved files are:

- `fft_overlap_x_axis_all_sensors.png`
- `fft_overlap_y_axis_all_sensors.png`
- `fft_overlap_z_axis_all_sensors.png`

22.3 FFT Plot by Sensor

The analyzer also creates one FFT plot per sensor with all three axes overlaid.

Saved files:

- `fft_plot_all_axes_s1.png`
- `fft_plot_all_axes_s2.png`
- `fft_plot_all_axes_s3.png`

23 Summary Text File

The analyzer writes a final text summary file:

```
summary_txt_path = run_folder / "summary_V2.txt"
```

23.1 Purpose

This file summarizes the entire analysis.

It includes:

- Raw file analyzed
- Processed folder path
- Data mode
- Sampling quality
- Estimated sampling rate
- Nyquist frequency
- Duration
- Number of samples
- Dropped sample estimate
- Results for each sensor-axis
- Dominant frequency
- RMS acceleration
- Peak acceleration
- Peak-to-peak acceleration

- Top FFT peaks
- Transmissibility summary
- List of saved files

23.2 Functions Used

`open()` Origin:

Python built-in function.

Purpose:

Opens the summary text file for writing.

`f.write()` Origin:

Python file object method.

Purpose:

Writes text into the summary file.

24 Final Terminal Output

At the end, the analyzer prints:

- Analysis complete message
- Processed folder path
- Summary file path
- Top peaks CSV path
- Frequency band summary path
- Sampling quality path
- Transmissibility results path, if applicable

This helps the user locate the generated results.

25 FFT Theory Summary

25.1 Time Domain

The measured acceleration signal is:

$$a(t)$$

This shows acceleration as a function of time.

25.2 Frequency Domain

The FFT converts the signal into:

$$A(f)$$

This shows acceleration magnitude as a function of frequency.

25.3 FFT Interpretation

A peak in the FFT means that the structure or sensor location contains vibration energy at that frequency.

Possible causes include:

- Motor excitation
- Printer motion frequency
- Frame resonance
- Belt or pulley excitation
- Harmonics of a forcing frequency
- Structural vibration modes

25.4 Dominant Frequency

The dominant frequency is the frequency with the largest FFT magnitude, excluding the zero-frequency component.

$$f_{\text{dominant}} = f [\arg \max M(f)], \quad f > 0$$

25.5 Nyquist Limit

The maximum reliable FFT frequency is:

$$f_N = \frac{f_s}{2}$$

Any vibration above this frequency may alias into the measured frequency range.

26 Complete Analyzer Execution Flow

The full analyzer process is:

1. Import required packages.
2. Define raw and processed data folders.

3. Define sensor and axis settings.
4. Define FFT peak and frequency-band settings.
5. Create processed data directory.
6. Search for raw CSV files.
7. Display available CSV files.
8. Let the user choose a file or use the latest file.
9. Create a unique processed run folder.
10. Load the selected CSV file.
11. Detect whether the CSV is three-sensor or legacy mode.
12. Verify required columns exist.
13. Copy the raw CSV into the processed folder.
14. Convert timestamps from microseconds to seconds.
15. Estimate the real sampling rate.
16. Calculate Nyquist frequency.
17. Calculate sampling statistics.
18. Check sample index for dropped samples.
19. Save sampling quality summary.
20. Plot sampling jitter.
21. Define sensor orientation corrections.
22. Define custom signal-processing functions.
23. Analyze every sensor-axis.
24. Save individual time-domain plots.
25. Save individual FFT plots.
26. Save top FFT peak table.
27. Save frequency-band summary table.
28. Save combined FFT CSV.
29. Save combined time-domain CSV.

30. Calculate transmissibility ratios.
31. Save transmissibility plots and CSV files.
32. Save combined time-domain plots.
33. Save combined FFT plots.
34. Write final text summary.
35. Print completion messages.

27 Summary

The analyzer script is responsible for transforming raw CSV vibration data into engineering results.

The most important custom functions are:

- `centered_signal()`
- `compute_fft()`
- `find_top_peaks()`
- `summarize_frequency_bands()`
- `analyze_column()`

The most important imported functions are:

- `pd.read_csv()` from Pandas
- `pd.DataFrame()` from Pandas
- `DataFrame.to_csv()` from Pandas
- `np.diff()` from NumPy
- `np.mean()` from NumPy
- `np.sqrt()` from NumPy
- `np.abs()` from NumPy
- `np.hanning()` from NumPy
- `np.fft.rfft()` from NumPy FFT
- `np.fft.rfftfreq()` from NumPy FFT
- `np.argmax()` from NumPy

- `np.maximum()` from NumPy
- `plt.plot()` from Matplotlib
- `plt.savefig()` from Matplotlib
- `shutil.copy2()` from Python's standard library
- `Path.glob()` from `pathlib`
- `Path.mkdir()` from `pathlib`

The analyzer estimates the real sampling rate from the timestamps instead of assuming the target ESP32 sampling rate. This is important because the FFT frequency axis depends directly on the actual sampling frequency.

The FFT implementation uses a Hanning window, real FFT, frequency-bin calculation, and magnitude normalization. The result is a vibration spectrum for each sensor-axis.

The analyzer also calculates transmissibility using sensor 3 as the reference input sensor and sensors 1 and 2 as response sensors.

Overall, `analyze_fft.py` converts raw experimental vibration data into meaningful plots, frequency peaks, energy summaries, and transmissibility results.